



Anexă

Tipuri de date · Structuri de control · Fișiere text · Funcții ·
Recursivitate

Andrei Mihai Ciobanu

Algolymp Summer School

Tipuri de date simple

Structuri de control

Fişiere text

Funcţii

Recursivitate

Resurse

Tipuri de date simple

Tipuri de date simple

Tip	Exemplu	Reprezintă
int, long long	7, -3	numere întregi
float, double	3.14, -0.5	numere reale (cu zecimale)
char	'a', '?'	un singur caracter
bool	true, false	valoare logică

```
1 int n = 7;  
2 double x = 3.14;  
3 char c = 'a';  
4 bool gasit = false;
```

▷ *tipuri simple de date (vezi slide-ul Resurse)*

int vs. long long

Tip	Biți	Interval
int	32	$\approx \pm 2.1 \cdot 10^9$
unsigned int	32	$0 \dots \approx 4.3 \cdot 10^9$
long long	64	$\approx \pm 9.2 \cdot 10^{18}$
unsigned long long	64	$0 \dots \approx 1.8 \cdot 10^{19}$

De obicei, dacă un calcul depășește $\approx 2 \cdot 10^9$, folosim long long. Variantele unsigned nu stochează valori negative. Limitele vin din reprezentarea în baza 2 — vezi *Lecția 3*.

▷ *Fun fact: float/double sunt stocate după standardul IEEE 754 — de aceea $0.1 + 0.2 \neq 0.3$ în orice limbaj de programare. Wikipedia: Floating-point arithmetic*

Structuri de control

Structura alternativă (if/else)

```
1  int x;  
2  cin >> x;  
3  
4  if (x > 0) {  
5      cout << "pozitiv";  
6  } else if (x < 0) {  
7      cout << "negativ";  
8  } else {  
9      cout << "zero";  
10 }
```

- Se execută un singur bloc, în funcție de condiție
- `else if` e opțional, poate apărea de mai multe ori
- `else` (fără condiție) e opțional, mereu la final

Structura repetitivă (for, while)

```
1 // for: cand stim de la inceput cate repetari facem
2 for (int i = 0; i < n; i++) {
3     cout << i << ' ';
4 }
5
6 // while: repetam cat timp e adevarata o conditie
7 int i = 0;
8 while (i < n) {
9     cout << i << ' ';
10    i++;
11 }
```

Ambele bucle fac același lucru aici — alegem forma potrivită contextului.

Structura repetitivă (do while)

Exemplu: câte cifre are numărul n ?

```
1 // while: n=0 => cifre=0
2 int cifre = 0;
3 while (n > 0) {
4     cifre++;
5     n /= 10;
6 }
```

```
1 // do while: pt n=0 => cifre
    =1
2 int cifre = 0;
3 do {
4     cifre++;
5     n /= 10;
6 } while (n > 0);
```

- Corpul se execută **cel puțin o dată**, indiferent de condiție
- Alternativă la **while**: un **if** ($n == 0$) $cifre = 1$; înaintea buclei

break și continue

```
1 for (int i = 0; i < 10; i++) {  
2     if (i == 3) continue; // sare la următoarea iteratie  
3     if (i == 7) break;    // iese din bucla  
4     cout << i << ' '  
5 }  
6 // afiseaza: 0 1 2 4 5 6
```

- **continue** — sare la următoarea iterație, fără a executa restul corpului
- **break** — iese din buclă imediat, indiferent de condiție
- Funcționează în **for**, **while** și **do while**

Fișiere text

De ce fișiere text?

Context

La majoritatea problemelor de olimpiadă (OJI, pbinfo, infoarena), datele se citesc dintr-un fișier `.in` și rezultatul se scrie într-un fișier `.out`, nu în consolă.

- Citire: `ifstream` (*input file stream*)
- Scriere: `ofstream` (*output file stream*)
- Se folosesc la fel ca `cin/cout`, doar că lucrează cu un fișier în loc de consolă

ifstream / ofstream

```
1  #include <fstream>
2  using namespace std;
3
4  int main() {
5      ifstream fin("date.in");
6      ofstream fout("date.out");
7      int a, b;
8      fin >> a >> b;
9      fout << a + b;
10     return 0;
11 }
```

- `fin >> a >> b;` citește la fel ca `cin`
- `fout << ...;` scrie la fel ca `cout`
- Fișierele se închid automat la finalul programului

Funcții

De ce folosim funcții?

Până acum am scris tot codul direct în `main`. Pe măsură ce problemele devin mai complexe, avem nevoie să **organizăm** codul.

O funcție:

- are un **nume**
- primește (opțional) **parametri**
- execută o secvență de instrucțiuni
- poate **întoarce** un rezultat (sau nu, dacă e `void`)

Avantaje:

- **Reutilizare:** scriem logica o singură dată, o folosim de câte ori avem nevoie
- **Claritate:** codul din `main` devine mai ușor de citit
- **Testare:** putem verifica fiecare funcție separat

Declarare, definire, apel

```
1 // declarare + definire: tip, nume, parametri
2 int suma(int a, int b) {
3     return a + b;
4 }
5
6 int main() {
7     int x = 3, y = 5;
8     int rez = suma(x, y); // apel
9     cout << rez;          // afiseaza 8
10    return 0;
11 }
```

- Tipul din fața numelui = tipul valorii întoarse (int, bool, void, ...)
- O funcție trebuie declarată **înainte** să fie folosită

Variabile locale și variabile globale

```
1  int contor = 0; // variabila globala, vizibila peste tot
2
3  void incrementeaza() {
4      int pas = 1; // variabila locala, doar in aceasta
                    functie
5      contor += pas;
6  }
7
8  int main() {
9      incrementeaza();
10     incrementeaza();
11     cout << contor; // afiseaza 2
12     return 0;
13 }
```

Parametri prin valoare vs. prin referință

```
1 void prinValoare(int x) {  
2     x = x + 10; // modifica doar copia locala  
3 }  
4  
5 void prinReferinta(int &x) {  
6     x = x + 10; // modifica variabila originala  
7 }  
8  
9 int main() {  
10     int a = 5, b = 5;  
11     prinValoare(a);  
12     prinReferinta(b);  
13     cout << a << ' ' << b; // afiseaza 5 15  
14     return 0;  
15 }
```

`int &x` primește **aceeași variabilă**, nu o copie.

Recursivitate

Ce este recursivitatea?

Definiție

O funcție este **recursivă** dacă se apelează pe ea însăși.

Orice funcție recursivă are nevoie de:

- Un **caz de bază** — condiția de oprire (altfel rulează la infinit)
- Un **apel recursiv** pe o subproblemă mai mică

Exemplu: $n! = n \cdot (n - 1)!$, cu cazul de bază $0! = 1$

▷ *Recursivitate — reluăm și aprofundăm la Lecția 2 (Algoritmul lui Euclid)*

Un prim exemplu: factorial

```
1 long long factorial(int n) {  
2     if (n == 0) {  
3         return 1;           // caz de baza  
4     }  
5     return n * factorial(n - 1); // apel recursiv  
6 }
```

- factorial(4) apelează factorial(3), care apelează factorial(2), ... până la factorial(0)
- Fiecare apel așteaptă rezultatul celui următor, apoi înmulțește
- Vom relua ideea, cu mai multe exemple, la *Lecția 2* (Algoritmul lui Euclid)

Resurse

Tipuri de date

- **pbinfo.ro** — Tipuri de date C/C++ (RO)
- **pbinfo.ro** — Intrări-ieșiri în C++ (RO)
- **pbinfo.ro** — Structura liniară (RO)
- **Wikipedia** — Floating-point arithmetic — IEEE 754 (EN)

Structuri de control

- **pbinfo.ro** — Structuri alternative (RO)
- **pbinfo.ro** — Structuri repetitive (RO)

Fișiere text

- [pbinfo.ro](#) — Operații de I/O cu fișiere în C++ (RO)
- [pbinfo.ro](#) — Fișiere text (RO)
- [geeksforgeeks.org](#) — File Handling through C++ Classes (EN)

Funcții | Recursivitate

- [pbinfo.ro](#) — Subprograme (RO)
- [pbinfo.ro](#) — Definirea și declararea unei funcții C/C++ (RO)
- [pbinfo.ro](#) — Recursivitate (RO)